

Comprehensive Audit and Fix Plan for a Misbehaving “Macrophage” Agent

Assumptions and scope

This report assumes **the repository URL is unspecified**, so I cannot directly clone, index, or point to exact files/lines in *your* codebase (“location in code” below is therefore expressed as **high-probability search targets** and **how to find the exact locations** quickly).

This report also assumes your **runtime environment is unspecified** (language, framework, deployment target, and whether the agent is single-process, multi-threaded, async, multi-process, containerized, or distributed). Where recommendations differ materially by runtime, I provide **alternatives** and **selection criteria** instead of a single prescriptive choice.

Finally, I assume the “macrophage” agent is a **software agent** (rule-based, heuristic, RL, or LLM-tool agent) with an initialization → run/step → shutdown lifecycle; the advice still applies if it is an “agent” in an ABM/simulation sense (because the failure modes—state leakage, nondeterminism, concurrency, brittle heuristics—are similar).

Deep-research sources are prioritized starting with **GitHub-based scanning and GitHub security features**, followed by **primary/official documentation** (language/runtime docs, observability standards, security standards) and a small number of widely used practitioner references. ¹

Executive summary

Intermittent “sometimes incorrect” agent behavior is most commonly caused by one or more of the following interacting classes of issues:

- **Nondeterminism** from uncontrolled randomness, hash randomization / iteration order, floating-point nondeterminism, or concurrency scheduling; this makes bugs appear “flaky” and hard to reproduce. Python’s `random` module explicitly notes reproducibility is contingent on conditions such as threading, and hashing/iteration order can vary across runs when hash randomization is involved. ²
- **Shared mutable state (state leakage)** across episodes/requests/sessions, especially when agents are reused, pooled, or run concurrently.
- **Cancellation and lifecycle bugs** in async systems (e.g., swallowing cancellation exceptions) that cause tasks to “half-fail” and leave the agent in an invalid state; Python’s `asyncio` docs explicitly warn structured concurrency can misbehave if cancellation is swallowed. ³
- **Brittle heuristics/decision logic** (thresholds, ordering, tie-breakers, fallback rules) that become unstable under minor distributional input changes or partial failures.

- **Insufficient observability** (lack of structured logs, traces, correlation IDs, deterministic replay artifacts), which prevents rapid identification of the exact decision path that produced the incorrect behavior. OpenTelemetry provides a standardized model for traces/metrics/logs correlation, while W3C Trace Context defines interoperable headers to propagate trace context. ⁴
- **Supply chain and dependency risks** that can change agent behavior or introduce vulnerabilities over time; GitHub Dependabot alerts/updates, dependency review, SBOM export, code scanning (CodeQL), and secret scanning are designed to surface these risks. ⁵

Given your goal (“thorough, prioritized audit and actionable fixes”), the fastest path to correctness is typically:

- 1) **Make the incorrect behavior reproducible** (seed discipline + deterministic replay + structured telemetry). ⁶
- 2) **Constrain state and lifecycle** with explicit invariants and a well-defined state machine.
- 3) **Harden concurrency boundaries** (TaskGroup/structured concurrency discipline, actor/message model, or strict locking) and eliminate shared mutable cross-agent state. ⁷
- 4) **Add CI gates** (lint/type/security/test/coverage) so the fixes remain stable. ⁸

GitHub-first repository scan strategy

Because the repository URL is missing, the “GitHub-first” section is written as a **copy/paste playbook** you can run immediately once you substitute your repo identifier (e.g., `ORG/REPO`) and (optionally) enable GitHub features in repo settings.

Index and triage everything that GitHub already knows

Use GitHub metadata to narrow the search before reading code:

- **Issues / PRs:** Extract bug reports mentioning “macrophage”, “agent”, “flaky”, “nondeterministic”, “race”, “seed”, “state leak”, “timeout”, “CancelledError”, “deadlock”, etc. GitHub’s API and `gh issue list` are designed for enumerating this quickly. ⁹
- **Workflow runs and logs:** Determine if CI failures correlate with the incorrect agent behavior (time-of-day, OS, Python/Node version, runner). `gh run list` and the Actions workflow run API support listing/rerunning and analyzing logs. ¹⁰
- **Security tab:** Check if you already have **Dependabot alerts**, **Dependabot security updates**, **secret scanning alerts**, and **code scanning alerts**. Secret scanning scans the full Git history for many secret types. ¹¹
- **Dependency graph and SBOM:** Export an SBOM (SPDX-compatible) and review dependency drift; GitHub supports SBOM export via UI and REST API. ¹²

Establish baseline security and quality scanning on GitHub

If not already enabled:

- **Code scanning via CodeQL:** Enable CodeQL (default or advanced). GitHub documents default vs advanced setup; advanced is preferred if you need custom build steps or custom query suites. ¹³

- **Use expanded query suites where appropriate:** The `security-extended` suite is broader than default and may surface additional issues (with more false positives). ¹⁴
- **Enable Dependabot alerts + security updates:** Dependabot can raise PRs to update vulnerable dependencies (including vulnerable GitHub Actions). ¹⁵
- **Enable dependency review on PRs:** This helps prevent introduction of vulnerable dependencies earlier than “alert later.” ¹⁶
- **Enable secret scanning:** It scans the entire Git history; GitHub also periodically rescans when new secret types are added. ¹⁷
- **Harden Actions credentials:** Apply least-privilege permissions for `GITHUB_TOKEN` and secrets; GitHub warns that repository write access implies read access to secrets, so workflow permissions should be minimized. ¹⁸

Suggested GitHub CLI command set for a first pass

```
# 0) Authenticate once
gh auth login

# 1) Repo metadata
gh repo view ORG/REPO --web

# 2) Issues/PRs that mention the agent
gh issue list -R ORG/REPO --search
"macrophage agent flaky nondeterministic seed race" --limit 200
gh pr list -R ORG/REPO --search "macrophage agent flaky nondeterministic seed
race" --limit 200

# 3) Recent CI runs to correlate with reported misbehavior
gh run list -R ORG/REPO --limit 50

# 4) Download logs for a specific run (replace RUN_ID)
gh run view -R ORG/REPO RUN_ID --log

# 5) Pull Security alerts (Dependabot/code scanning) via web UI:
# https://github.com/ORG/REPO/security
```

This aligns with the GitHub CLI’s documented capabilities to list runs and issues. ¹⁹

What “linked GitHub resources” typically means in practice

When users say “scan linked GitHub resources,” the main hidden risk is **transitive trust**:

- Git submodules, vendored code, and copied snippets.
- Reusable workflows and third-party GitHub Actions referenced in `.github/workflows/*.yaml`.
- External repos referenced in docs (examples, “copy this config,” etc.).

Dependabot security updates explicitly cover vulnerable GitHub Actions when enabled. ²⁰
 CodeQL has query help for GitHub Actions workflows, and GitHub documents query suites/coverage for scanning multiple languages and CI definitions. ²¹

Prioritized issue register for the macrophage agent

Because the repo is not available, the table below is a **prioritized issue register** designed to match the symptom “sometimes behaves incorrectly.” “Location” is given as **where it almost always lives** and **how to pinpoint it** rapidly.

Prioritized issue register

Priority	Issue theme	Severity	Likelihood	Typical symptoms	Likely location in code	Fast way to find exact spot
P0	Uncontrolled randomness and missing seed discipline	Critical	Very high	Same inputs → different actions; flaky tests; “works on my machine”	Agent policy/heuristics; sampling; exploration; tie-breakers; any <code>random</code> / <code>np.random</code> / model sampling params	<code>rg -n "random\ \. np\\.random seed temperature top_p sample"</code>
P0	Shared mutable state leaking across runs/episodes	Critical	Very high	Agent “remembers” wrong things; diverges after one failure; cross-request contamination	<code>MacrophageAgent</code> fields; module-level caches; singletons; “global” registries; static class vars	<code>rg -n "global singleton cache static classvar lru_cache @cache"</code>
P0	Concurrency race conditions around state updates	Critical	High	Rare incorrect decisions; order-dependent behavior; data structures in invalid intermediate states	Shared state store; async tool calls; callbacks; concurrent handlers	<code>rg -n "async await create_task gather Thread Process lock Semaphore Queue"</code>
P0	Async cancellation swallowed or mishandled	High	High (if async)	Timeouts cause corrupted state; tasks “complete” but agent is broken	Broad exception handlers around awaits; task supervisors	Search for <code>CancelledError</code> handling; ensure not swallowed (see guidance below)

Priority	Issue theme	Severity	Likelihood	Typical symptoms	Likely location in code	Fast way to find exact spot
P1	Input preprocessing mismatch and schema drift	High	Medium-High	Incorrect behavior only on certain inputs; fails silently; "garbage in" becomes "confident wrong"	Parsers, normalizers, feature builders, prompt templates	<code>rg -n "parse normalize preprocess schema json \.loads pydantic validator"</code>
P1	Iteration order dependence (hashing, sets, dict ordering assumptions)	High	Medium	Rare reordering changes lead to different "best" choice	Ranking logic; "first match wins"; set/dict iteration	<code>rg -n "set\\(dict\\(sorted\\(min\\(max\\(tie first break"</code>
P1	Heuristic brittleness / thresholds without calibration	Medium-High	High	Agent flips decision around boundary; inconsistent across distributions	Scoring weights; thresholds; "if score > x"	<code>rg -n "threshold score\\s*[<>]= weight epsilon margin"</code>
P2	Observability gaps (no correlation IDs, no decision trace)	Medium	Very high	"Cannot reproduce"; low confidence root cause analysis	Logging wrappers; agent loop; tool execution	<code>rg -n "logger log trace span metric telemetry"</code>
P2	Performance/memory regressions causing timeouts or partial execution	Medium	Medium	Stalls, timeouts, OOM → incorrect action or fallback path	Memory stores; history buffers; embedding caches; large objects retained	Profile + leak checks (tools below)
P2	Dependency drift / security issues changing behavior	Medium-High	Medium	Behavior changes after dependency upgrade; vulnerable packages	<code>requirements*.txt</code> , <code>pyproject.toml</code> , <code>package.json</code> , lockfiles	SBOM + Dependabot + pip/npm audits

Priority	Issue theme	Severity	Likelihood	Typical symptoms	Likely location in code	Fast way to find exact spot
P3	Documentation/ onboarding gaps	Medium	High	New devs cannot run agent; “mystery config”; inconsistent env	README, runbooks, examples, configuration docs	Search for missing “quickstart” or undocumented env vars

This prioritization is grounded in the documented fragility of reproducibility under threading and hash randomization in Python, and in the known ways structured concurrency can misbehave when cancellation is mishandled. ²²

Concrete code-level fixes and refactors

This section provides **concrete, code-level patterns** that reliably fix intermittent agent misbehavior across languages/frameworks. Pick the smallest set that address your likely P0/P1 causes first.

Make the macrophage agent an explicit state machine with a tight lifecycle

A common architecture failure is letting the agent be an “object with methods” whose valid operations are implicit. A robust alternative is:

- Define explicit lifecycle states: `CREATED → INITIALIZED → READY → RUNNING → STOPPING → STOPPED → ERROR`.
- Enforce allowed transitions.
- Separate pure decision logic from side effects.

Mermaid lifecycle diagram:

```
stateDiagram-v2
    [*] --> CREATED
    CREATED --> INITIALIZED: init(config, deps)
    INITIALIZED --> READY: warmup()
    READY --> RUNNING: start(run_id)
    RUNNING --> RUNNING: step(observation)
    RUNNING --> STOPPING: stop()
    STOPPING --> STOPPED: cleanup()
    RUNNING --> ERROR: invariant_violation / exception
    ERROR --> STOPPING: best-effort cleanup()
    STOPPED --> [*]
```

Why this helps: it localizes “sometimes incorrect” behavior to either (a) decision logic that violated an invariant, or (b) invalid ordering of lifecycle calls.

Refactor decision logic into a pure core with injected dependencies

The most corrective refactor for intermittent behavior is a “functional core, imperative shell” pattern:

- **Core:** `decide(state, observation, rng) -> (new_state, action, debug_events)`
- **Shell:** runs tools, handles I/O, serialization, concurrency, timeouts, logging.

A minimal Python sketch (adapt to your language):

```
from dataclasses import dataclass, replace
from typing import Any, Dict, Optional, Tuple
import random

@dataclass(frozen=True)
class MacrophageState:
    episode_id: str
    step_idx: int
    memory: Tuple[str, ...]
    last_action: Optional[str] = None

@dataclass(frozen=True)
class MacrophageConfig:
    exploration_rate: float
    max_memory: int
    seed: int

class MacrophageAgent:
    def __init__(self, cfg: MacrophageConfig) -> None:
        self.cfg = cfg
        self.rng = random.Random(cfg.seed) # per-agent RNG (no global random)
        self.state: Optional[MacrophageState] = None

    def init_episode(self, episode_id: str) -> None:
        self.state = MacrophageState(episode_id=episode_id, step_idx=0,
memory=())

    def decide(self, observation: Dict[str, Any]) -> str:
        assert self.state is not None, "Agent not initialized"
        s = self.state

        # Example: deterministic tie-breaking via RNG under explicit exploration
        explore = (self.rng.random() < self.cfg.exploration_rate)
        if explore:
            action = "RANDOM_EXPLORE"
        else:
            action = self._best_action(observation, s)
```

```

        # Update state immutably
        new_mem = (s.memory + (f"obs:{observation.get('type')}",))[-
self.cfg.max_memory :]
        self.state = replace(s, step_idx=s.step_idx + 1, memory=new_mem,
last_action=action)
        return action

    def _best_action(self, observation: Dict[str, Any], state: MacrophageState)
-> str:
        # pure rule/heuristic scoring here
        return "PHAGOCYTOSE" if observation.get("threat_score", 0) > 0.7 else
"PATROL"

```

Key principles embedded here:

- **Per-agent RNG** (not module-global) eliminates a major source of cross-test cross-episode nondeterminism. Python's docs note reproducibility depends on conditions such as multi-threading; per-agent RNG also makes it easier to log seeds and replay. ²³
- **Immutable state updates** reduce accidental mutation and make "state snapshots" loggable and replayable.
- A single entry point (`decide`) plus explicit `init_episode` makes lifecycle misuse obvious.

If you use NumPy RNG, prefer dedicated `Generator` instances and `SeedSequence` for safe parallel streams rather than reseeding global singleton RNGs. ²⁴

Concurrency hardening: choose one of three patterns and enforce it

Intermittent incorrect behavior frequently appears when agent state can be updated concurrently (even if you "think" it can't). The fix is to choose a concurrency model and enforce it rather than letting concurrency emerge ad hoc.

Concurrency model	How it prevents incorrect behavior	Pros	Cons	When to choose
Single-threaded agent loop + async I/O	State updated in one place; async tasks return results, but state mutation is serialized	Simplest reasoning; fewer races	Might reduce throughput	Most agent architectures; easiest correctness
Actor/message model	Agent owns its state; external callers send messages; mailbox serializes updates	Strong isolation; scales to multiple agents well	Requires refactor; message discipline	Multi-agent systems; high concurrency needs

Concurrency model	How it prevents incorrect behavior	Pros	Cons	When to choose
Locks around state store	Protect shared state with <code>Lock/RWLock</code>	Minimal refactor	Easy to deadlock; hard to audit; performance costs	Only if you must share state across threads

For Python async code, structured concurrency patterns (like `TaskGroup`) exist specifically to avoid orphaned tasks and unclear cancellation behavior; but they can misbehave if cancellation is mishandled, so you must not swallow `CancelledError` in broad exception blocks. ³

Data flow separation and validation at boundaries

Most agent bugs are amplified by weak boundaries between:

- raw input → preprocessing → features/prompts → decision logic → action/tool execution → state update

Mermaid data flow diagram:

```

flowchart LR
    A["A[Raw Inputs\n(events, user text, sensors)]"] --> B["B[Validation & Schema\n(parse, normalize)]"]
    B --> C["C[Feature/Context Builder\n(history, memory, tools context)]"]
    C --> D["D[Decision Core\n(policy / heuristics / LLM prompt)]"]
    D --> E["E[Action Executor\n(tool calls, env updates)]"]
    E --> F["F[State Update\n(commit new state + audit record)]"]
    F --> C
    F --> G["G[Telemetry\n(logs + traces + metrics)]"]

```

Implementation guidance:

- Put **schema validation** immediately after parsing (reject or quarantine invalid inputs rather than “best-effort” coercion).
- Make preprocessing **pure and deterministic** (no random sampling, no time-dependent fallbacks).
- Ensure the decision core cannot access raw unvalidated inputs directly.

Randomness and seeding: a reproducibility contract

A robust reproducibility contract typically includes:

- A single **run identifier** + **global master seed** recorded in logs/traces.
- Derived **per-agent seeds** (`seed = hash(master_seed, agent_id, episode_id)` or `SeedSequence.spawn`) rather than one global RNG shared by all agents.

- Declaring whether behavior is expected to match **across machines** or only **within machine**; some libraries warn that seeding alone may not make all operations deterministic (especially with GPU kernels), and PyTorch provides deterministic-algorithms modes for this reason. ²⁵
- Avoid reliance on iteration order of hash-based collections to break ties; Python hash randomization affects set iteration ordering and can change between invocations. ²⁶

Tests, CI checks, and reproducible debugging

Testing strategy that directly targets “sometimes incorrect”

A useful test pyramid for agent correctness differs from typical CRUD apps: you want *replay*, *property checks*, and *concurrency stress*.

Test type	What it catches	Suggested tooling examples	How to make it reproducible
Deterministic replay tests	Regression in decision logic; rare edge-case sequences	Golden traces; snapshot tests; recorded event streams	Fix seed + freeze inputs + assert action sequence
Property-based tests	Unexpected behavior across wide input space	Hypothesis (Python) generates varied inputs/edge cases	Save failing examples; Hypothesis supports replaying failures ²⁷
Concurrency stress tests	Races, ordering issues	Thread/async stress harnesses; randomized scheduling	Run many iterations; assert invariants
Integration tests	Tool/environment coupling issues	End-to-end tests with mocked tools	Record tool I/O; do not hit live services
Fuzz / schema fuzz	Parser/preprocess bugs	Structured fuzzing for JSON/schemas	Keep corpus + seeds

Hypothesis is explicitly designed to generate broad input coverage and preserve/replay failing examples, which is particularly effective for intermittent agent bugs caused by boundary conditions. ²⁷

CI checks that should be non-negotiable

Even without knowing your language, the CI *categories* are stable:

- **Lint + format** (fast feedback)
- **Static type / static analysis** (catches state shape mismatches early)
- **Unit + property + replay tests**
- **Coverage** (to prevent “tests exist but don’t touch agent core”)
- **Security checks** (dependencies + secrets + code scanning)

GitHub Actions workflows are YAML-defined and support matrix strategies to test across language/runtime versions, and can also control concurrency to avoid overlapping deployments/tests that would interfere with artifacts. ²⁸

Example CI command bundle (Python-oriented; adapt as needed):

```
# Lint/format
ruff check .
ruff format --check .

# Types (choose one)
mypy .
pyright

# Tests + coverage
pytest -q
coverage run -m pytest
coverage report -m

# Dependency security (Python & Node examples)
pip-audit
npm audit
```

Ruff is configured via `pyproject.toml` / `ruff.toml`; mypy and pyright are widely used static type checkers; coverage.py measures executed lines/branches; npm audit reports dependency vulnerabilities for Node projects. ²⁹

GitHub-native security checks to wire into CI

- **CodeQL code scanning** (default/advanced). ¹³
- **Dependabot alerts + security updates PRs**. ³⁰
- **Dependency review** on PRs. ³¹
- **Secret scanning** (history-wide). ¹⁷
- **SBOM export** for auditing and incident response. ¹²

Reproducible debugging steps that converge quickly

The following sequence tends to reliably convert “sometimes wrong” into an actionable root cause:

- 1) **Freeze a failing trace**: log the run ID, episode ID, agent ID, seed(s), and a canonical serialized observation sequence.
- 2) **Add decision journaling**: for each step, record inputs → derived features → chosen action → reason code → updated state hash.
- 3) **Introduce invariant checks**: fail fast when state violates constraints (memory size, null fields, monotonic counters, valid lifecycle transitions).
- 4) **Binary search regressions** with `git bisect` once you have a deterministic reproducer; `git bisect`

is explicitly designed to locate the commit introducing a bug via binary search. ³²

5) **Pinpoint ownership** with `git blame` once the failing line is identified. ³³

Example (high-leverage) debugging commands:

```
# Find the precise commit that introduced the regression (once a reproducer
exists)
git bisect start
git bisect bad HEAD
git bisect good <KNOWN_GOOD_TAG_OR_COMMIT>

# Then repeatedly run the reproducer script and mark good/bad:
# git bisect good
# git bisect bad

# Search codebase quickly for macrophage agent and its state mutations
rg -n "Macrophage|macrophage|agent_state|state\\.|set_state|update_state" .
```

`ripgrep` is purpose-built for fast recursive search and respects `.gitignore` by default, making it ideal for locating “state mutation” hotspots. ³⁴

Performance, memory, logging, observability, and security hardening

Logging and observability that supports deterministic replay

A correctness-oriented observability baseline for agents typically includes:

- **Structured logs** with stable keys: `run_id`, `episode_id`, `agent_id`, `step`, `seed`, `action`, `policy_version`, `input_hash`, `state_hash`.
- **Trace correlation** across tool calls and agent steps (especially if the agent calls external services). W3C Trace Context defines vendor-neutral propagation headers (`traceparent`, `tracestate`). ³⁵
- **OpenTelemetry** for standardized traces/metrics/logs (and correlation across signals); OpenTelemetry’s spec and docs describe the collector and signal model. ³⁶
- **Treat logs as an event stream** (do not rely on local files as the system of record). ³⁷

If your macrophage agent is LLM-based or consumes untrusted text, incorporate prompt-safety controls and log *policy decisions* rather than leaking sensitive prompts. OWASP’s GenAI/LLM guidance includes prompt injection risk descriptions and prevention patterns. ³⁸

Performance and memory diagnostics

Intermittent incorrect behavior is sometimes a second-order effect of performance problems (timeouts, partial execution, fallback logic). For Python specifically:

- `cProfile` /profilers provide deterministic profiling of Python programs. ³⁹
- `tracemalloc` traces memory allocations and can diff snapshots to detect leaks. ⁴⁰
- Python also supports enabling `tracemalloc` via environment variable for debugging. ⁴¹

Example Python profiling harness:

```
python -m cProfile -o profile.pstats path/to/run_macrophage.py
python -c "import pstats; p=pstats.Stats('profile.pstats');
p.sort_stats('cumtime').print_stats(40)"
```

Example memory snapshot harness (conceptual):

```
import tracemalloc

tracemalloc.start()
# run agent workload
snap1 = tracemalloc.take_snapshot()
# run again or after many steps
snap2 = tracemalloc.take_snapshot()
top = snap2.compare_to(snap1, 'lineno')[:20]
for stat in top:
    print(stat)
```

These tools exist specifically to identify hot paths and allocation sources at line-level granularity. ⁴²

Security and dependency risk controls

A practical security baseline for agent codebases combines GitHub-native controls with ecosystem-specific audits:

- **Dependabot alerts + security updates** for vulnerable dependencies. ³⁰
- **Dependency review** to catch risky dependency changes at PR time (prevention over remediation). ³¹
- **SBOM export** for compliance and incident response; GitHub supports SPDX-compatible SBOM export. ¹²
- **Secret scanning** to prevent leaks of API keys/tokens (history-wide). ⁴³
- **Code scanning with CodeQL** (and consider `security-extended` for broader coverage). ⁴⁴
- **Supply chain hardening frameworks** (SLSA) can guide maturity improvements when you need attestations and stronger provenance. ⁴⁵

- For Python, `pip-audit` scans environments/requirements against vulnerability databases; for Node, `npm audit` reports known vulnerabilities and remediations. ⁴⁶

If the macrophage agent uses LLMs and tools, treat the system as a “GenAI application” for threat modeling: OWASP’s LLM Top 10 enumerates risks like prompt injection and supply chain vulnerabilities. ⁴⁷

Documentation and onboarding deliverables that prevent reintroduction of bugs

Correctness fixes decay unless the project makes the “right way” the easiest way:

- A **single Quickstart** that runs a deterministic local scenario (fixed seed + small scenario corpus).
- A **runbook**: “How to reproduce a bug from logs,” including required env vars and example commands.
- An **architecture note**: state model, lifecycle, concurrency model, and what is allowed to mutate state.
- **Pre-commit hooks** so formatting/linting is enforced before CI; `pre-commit install` and `pre-commit run --all-files` are the canonical workflow. ⁴⁸

Proposed implementation timeline and effort estimate

Because the repo and environment are unknown, estimates are ranges. The intent is to provide a realistic plan that prioritizes correctness and observability before heavy refactors.

Phase goals and estimated effort

Phase	Primary goal	Deliverables	Typical effort
Baseline instrumentation and repro	Turn “sometimes wrong” into a deterministic reproducer	Run IDs + seeds; decision journal; invariant checks; minimal replay harness	2–5 engineer-days
GitHub security and CI baselining	Prevent obvious regressions and supply-chain drift	CodeQL enabled; Dependabot alerts/updates; dependency review; secret scanning; CI matrix	1–3 engineer-days ⁴⁹
State/lifecycle refactor	Stop state leakage and lifecycle misuse	Explicit lifecycle state machine; immutable state updates or event log; per-episode reset guarantees	3–10 engineer-days
Concurrency hardening	Remove races and cancellation corruption	Single writer model or actor model; TaskGroup discipline; no swallowed cancellation	3–8 engineer-days ³
Test suite expansion	Lock in fixes and cover edge cases	Replay tests; property-based tests; stress tests; coverage gating	3–8 engineer-days ⁵⁰

Phase	Primary goal	Deliverables	Typical effort
Performance and memory pass	Eliminate timeout-driven incorrect behavior	Profiling report; memory snapshots; fixed leaks/hot paths; budgets	2–6 engineer-days ⁴²
Documentation and onboarding	Make correctness the default	Quickstart, runbook, architecture notes, pre-commit setup	1–3 engineer-days ⁴⁸

What to do first if you only have one week

If you can only invest ~5 engineer-days initially, the highest ROI sequence is:

- 1) Add replay artifacts + seeds + decision journaling (so the bug is not “intermittent” anymore). ⁵¹
- 2) Add invariant checks + explicit lifecycle transitions (fail fast, not silently).
- 3) Add property-based tests for the decision core (Hypothesis-style) and record failing examples. ²⁷
- 4) Enable CodeQL + Dependabot + secret scanning + dependency review (fast safety rails). ⁵²

Once the repository URL is known, the same framework can be applied to produce **fully concrete** findings with exact file paths, line-level hotspots, and a repo-specific remediation PR plan, using the GitHub and tooling workflows described above.

-
- 1 8 13 44 49 52 About code scanning with CodeQL
https://docs.github.com/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning-with-codeql?utm_source=chatgpt.com
- 2 6 22 23 51 Generate pseudo-random numbers
https://docs.python.org/3/library/random.html?utm_source=chatgpt.com
- 3 7 Coroutines and Tasks
https://docs.python.org/3/library/asyncio-task.html?utm_source=chatgpt.com
- 4 36 Overview
https://opentelemetry.io/docs/specs/otel/overview/?utm_source=chatgpt.com
- 5 15 20 About Dependabot security updates
https://docs.github.com/en/code-security/concepts/supply-chain-security/about-dependabot-security-updates?utm_source=chatgpt.com
- 9 gh issue list
https://cli.github.com/manual/gh_issue_list?utm_source=chatgpt.com
- 10 19 gh run list
https://cli.github.com/manual/gh_run_list?utm_source=chatgpt.com
- 11 30 About Dependabot alerts
https://docs.github.com/code-security/dependabot/dependabot-alerts/about-dependabot-alerts?utm_source=chatgpt.com

12 Exporting a software bill of materials for your repository

https://docs.github.com/en/code-security/how-tos/secure-your-supply-chain/establish-provenance-and-integrity/exporting-a-software-bill-of-materials-for-your-repository?utm_source=chatgpt.com

14 CodeQL query suites

https://docs.github.com/en/code-security/concepts/code-scanning/codeql/codeql-query-suites?utm_source=chatgpt.com

16 31 About dependency review

https://docs.github.com/code-security/supply-chain-security/understanding-your-software-supply-chain/about-dependency-review?utm_source=chatgpt.com

17 43 About secret scanning

https://docs.github.com/code-security/secret-scanning/about-secret-scanning?utm_source=chatgpt.com

18 Secure use reference - GitHub Docs

https://docs.github.com/en/actions/reference/security/secure-use?utm_source=chatgpt.com

21 CodeQL query help documentation - GitHub

https://codeql.github.com/codeql-query-help/?utm_source=chatgpt.com

24 numpy.random.seed — NumPy v2.2 Manual

https://numpy.org/doc/2.2/reference/random/generated/numpy.random.seed.html?utm_source=chatgpt.com

25 Reproducibility — PyTorch 2.9 documentation

https://docs.pytorch.org/docs/stable/notes/randomness.html?utm_source=chatgpt.com

26 41 1. Command line and environment

https://docs.python.org/3/using/cmdline.html?utm_source=chatgpt.com

27 50 Hypothesis 6.151.9 documentation

https://hypothesis.readthedocs.io/?utm_source=chatgpt.com

28 Workflow syntax for GitHub Actions

https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions?utm_source=chatgpt.com

29 Configuring Ruff - Astral Docs

https://docs.astral.sh/ruff/configuration/?utm_source=chatgpt.com

32 Git - git-bisect Documentation

https://git-scm.com/docs/git-bisect?utm_source=chatgpt.com

33 Git - git-blame Documentation

https://git-scm.com/docs/git-blame?utm_source=chatgpt.com

34 ripgrep recursively searches directories for a regex pattern ...

https://github.com/BurntSushi/ripgrep?utm_source=chatgpt.com

35 Trace Context

https://www.w3.org/TR/trace-context/?utm_source=chatgpt.com

37 Treat logs as event streams

https://12factor.net/logs?utm_source=chatgpt.com

38 LLM01:2025 Prompt Injection

https://genai.owasp.org/llmrisk/llm01-prompt-injection/?utm_source=chatgpt.com

39 **The Python Profilers**

https://docs.python.org/3/library/profile.html?utm_source=chatgpt.com

40 42 **tracemalloc — Trace memory allocations**

https://docs.python.org/3/library/tracemalloc.html?utm_source=chatgpt.com

45 **SLSA • Security levels**

https://slsa.dev/spec/v1.0/levels?utm_source=chatgpt.com

46 **pip-audit**

https://pypi.org/project/pip-audit/?utm_source=chatgpt.com

47 **OWASP Top 10 for Large Language Model Applications**

https://owasp.org/www-project-top-10-for-large-language-model-applications/?utm_source=chatgpt.com

48 **pre-commit**

https://pre-commit.com/?utm_source=chatgpt.com